

Tenet API Reference

- *version: 1.0*

1. 框架介绍

- **Tenet** 是一个基于 **C++** 的简易深度学习框架，目前仅支持 **多层感知机** 的开发
 - 使用 **Tenet** 需要 **C++11** 标准支持
 - **主要功能** 为快速搭建神经网络，自动求导并使用优化器训练模型
 - 使用此框架能实现 **数据预测**，**图像分类**等机器学习任务
 - **Tenet** 框架由三个头文件实现，分别为 **tensor.hpp** / **tenet.hpp** / **tools.hpp**
-

2. 快速上手

导入框架

使用框架只需要导入头文件 **tenet.hpp** 即可
头文件中包含的命名空间为 **nn** 和 **tools**，如有需要可声明

```
#include "tenet.hpp"
using namespace nn;
```

模型搭建

1. 定义Module类

搭建模型首先需要定义一个继承自 **Module类** 的子类

请注意： Module类定义在命名空间 **nn** 下，且继承时必须使用 **公有继承**，以下所有内容也必须定义为 **public**

2. 定义并初始化层 (Layer)

- 层 (Layer) 包含了全连接层以及各类激活函数
- **Linear 全连接层**，初始化时须在括号中定义 **input_size** 和 **output_size**
- **Relu / Sigmoid 激活函数** 初始化时无需在括号内定义内容

这些层同样来自命名空间 **nn**，它们也是类

有关其更细节的名称，定义以及初始化方法请转至第三节 **核心组件** 参考

3. 定义构造函数 (初始化函数)

我们需要在构造函数中使用 **register_layer** 函数来注册我们的层

- **使用方法：** 只需要在 **register_layer** 函数的括号中传入先前定义的层即可

**注：只有在层被注册后，Module类才可以读取Layer中存储参数以及梯度的地址
全连接层（含有参数以及梯度的层）必须被注册，激活函数用户可自由选择是否注册*

4. 重写forward函数

Tenet 在不同 **layer** 层传参时使用一个名为 **Telo** 的数据类型
故forward函数的定义方式为

```
Telo forward(Telo x);
```

Module 类的 **forward** 函数传入一个 **Telo**，并返回一个 **Telo**
我们需要在 **forward** 函数里定义前向传播过程

- **layer** 层的 **forward** 方法：传入一个 **Telo**（输入），返回一个 **Telo**（计算结果）

示例代码

```
class Example_Model : public nn::Module
{
public:
    nn::Linear fc1 = nn::Linear(5, 10);
    nn::Linear fc2 = nn::Linear(10, 1);
    nn::ReLU relu = nn::ReLU();
    Example_Model() {
        register_layer(fc1);
        register_layer(fc2);
        register_layer(relu);
    }
    Telo forward(Telo x) {
        x = fc1.forward(x);
        x = relu.forward(x);
        x = fc2.forward(x);
        return x;
    }
};
```

训练与评估

1. 声名模块

我们一共需要三个模块，分别为 **模型**，**损失函数** 和 **优化器**

Tenet 提供 **MSELoss** 和 **CrossEntropyLoss** 两种 **损失函数** 以及支持权重衰退的 **SGD** 优化器

```
My_Model model;
nn::MSELoss criterion;
nn::SGD optimizer(model.parameters(), 1r);
```

以上代码给出了标准的声名方法，其中优化器需要传入两个参数，分别为模型的参数以及学习率，我们可以通过 **Module**类的 **parameters**方法来获取模型的参数

2. 训练

我们将先给出单次训练的代码以供参考：

```
Telo outputs = model(features);
double loss = criterion(outputs, labels);
optimizer.zero_grad();
criterion.backward();
optimizer.step();
```

1. 首先在model的括号中传入输入数据，即可获得模型的返回值，类型为Telo
2. 在损失函数的括号中分别传入输出和标签，即可获得double类型的损失值，其中我们规定labels应为[batch_size, 1]大小的vector数组，对于分类问题，其值应为类别的索引（注：在tools工具中含有快速打包label并整理其格式的方法）
3. 之后使用优化器的zero_grad方法清空梯度，防止梯度累加
4. 使用损失函数的backwards方法进行反向传播，**注意！ Tenet1.0** 版本和主流框架不同，backward并非作用在损失值loss上
5. 最后使用优化器的step方法更新梯度

3. 核心组件

Tensor

Tensor 是 Tenet 框架的核心数据结构，用于存储和操作矩阵数据。它支持常见的矩阵运算、广播机制以及函数式操作。以下是其主要功能和使用方法：

构造函数

- `Tensor(int rows, int cols, double val = 0.0)`
创建一个大小为 `rows x cols` 的矩阵，并初始化为 `val`。
示例：

```
Tensor t1(3, 4, 1.0); // 创建一个 3x4 的矩阵，所有元素初始化为 1.0
```

- `Tensor(const std::vector<std::vector<double>> &M)`
使用二维向量 `M` 初始化 Tensor。
示例：

```
std::vector<std::vector<double>> data = {{1, 2}, {3, 4}};
Tensor t2(data); // 使用二维向量初始化
```

常用方法

- `size(int dim)`
返回指定维度的大小，`dim=0` 返回行数，`dim=1` 返回列数。
示例：

```
int rows = t1.size(0); // 获取行数
int cols = t1.size(1); // 获取列数
```

- `size()`

返回一个 `std::pair<int, int>`，表示行数和列数。

示例：

```
auto [rows, cols] = t1.size(); // 获取行数和列数
```

- `T()`

返回当前矩阵的转置。

示例：

```
Tensor t3 = t1.T(); // 获取转置矩阵
```

- `apply(std::function<double(double)> func)`

对矩阵中的每个元素应用函数 `func`，并直接修改当前矩阵。

示例：

```
t1.apply([](double x) { return x * 2; }); // 将矩阵中每个元素乘以 2
```

- `after(std::function<double(double)> func)`

对矩阵中的每个元素应用函数 `func`，并返回一个新的矩阵。

示例：

```
Tensor t4 = t1.after([](double x) { return x + 1; }); // 返回每个元素加 1 的新矩阵
```

工具函数

- `zeros(int rows, int cols)`

返回一个全零矩阵。

示例：

```
Tensor t5 = zeros(3, 3); // 创建一个 3x3 的全零矩阵
```

- `ones(int rows, int cols)`

返回一个全一矩阵。

示例：

```
Tensor t6 = ones(2, 2); // 创建一个 2x2 的全一矩阵
```

- `rands(int rows, int cols, double range = 0.5)`

返回一个随机矩阵，值在 `[-range, range]` 之间。

示例：

```
Tensor t7 = rands(4, 4, 1.0); // 创建一个 4x4 的随机矩阵，值范围为 [-1.0, 1.0]
```

Neural Network

Tenet 提供了多种神经网络组件，包括全连接层、激活函数和损失函数。以下是它们的功能和使用方法：

Layer

`Layer` 是所有神经网络层的基类，定义了 `forward` 和 `backward` 方法：

- `forward(Telo x)`

接收输入并返回输出。

示例：

```
Telo output = layer.forward(input);
```

- `backward(Tensor grad)`

接收梯度并计算上一层的梯度。

示例：

```
layer.backward(grad);
```

Linear

全连接层，构造时需要指定输入和输出的大小：

```
nn::Linear fc(5, 10); // 输入大小为 5，输出大小为 10
```

- `forward(Telo x)`

计算 $x * W + b$ ，其中 `W` 和 `b` 是参数。

示例：

```
Telo output = fc.forward(input);
```

- `backward(Tensor grad)`

计算梯度并更新参数。

激活函数

- **ReLU**

实现 ReLU 激活函数，使用方法与 `Linear` 类似。

示例：

```
nn::ReLU relu;  
Telo output = relu.forward(input);
```

- **Sigmoid**

实现 Sigmoid 激活函数，使用方法与 `ReLU` 类似。

损失函数

- **MSELoss**

均方误差损失函数，适用于回归任务。

示例：

```
nn::MSELoss criterion;  
double loss = criterion(outputs, labels);
```

- **CrossEntropyLoss**

交叉熵损失函数，适用于分类任务。

示例：

```
nn::CrossEntropyLoss criterion;  
double loss = criterion(outputs, labels);
```

Tools

`tools` 命名空间提供了一些实用工具函数，主要用于数据处理和调试。

DataLoader

`DataLoader` 是一个数据加载工具，用于将数据分批次加载，并支持随机打乱数据。

函数签名：

```
std::vector<std::pair<Tensor, Tensor>> DataLoader(  
    std::vector<std::pair<std::vector<double>, std::vector<double>>> data,  
    int batch_size,  
    bool shuffle = false  
);
```

- **参数：**

- `data`：一个包含特征和标签的向量，每个元素是一个 `std::pair`，其中 `first` 是特征，`second` 是标签。
- `batch_size`：每个批次的大小。
- `shuffle`：是否随机打乱数据，默认为 `false`。

- **返回值：**

返回一个向量，每个元素是一个批次的数据，包含特征和标签的 `Tensor`。

- **实现方法：**

1. 如果 `shuffle` 为 `true`，使用 `std::shuffle` 随机打乱数据。
2. 将数据按 `batch_size` 分割，每个批次的特征和标签分别打包成 `Tensor`。

- **示例：**

```
std::vector<std::pair<std::vector<double>, std::vector<double>>> data = {
    {{1.0, 2.0}, {0.0}},
    {{3.0, 4.0}, {1.0}},
    {{5.0, 6.0}, {0.0}}
};
auto batches = tools::DataLoader(data, 2, true);
for (const auto &batch : batches) {
    Tensor features = batch.first;
    Tensor labels = batch.second;
    print(features);
    print(labels);
}
```

print

`tools` 提供了多种打印方法，方便调试：

- `print(Tensor X)`

打印 Tensor 的内容。

示例：

```
Tensor t = ones(2, 2);
print(t);
```

- `print(std::pair<int, int> P)`

打印 Tensor 的大小。

示例：

```
print(t.size());
```

- `print(double x)`

打印标量值。

示例：

```
print(3.14);
```